

A Theoretical Foundation for the Notion-Based Reasoning System (NBRS)

Compositional Execution, Dependency Scheduling, and the Path Toward
Autonomous Reasoning

NBRS Working Group

May 17, 2026

Abstract

We develop the theoretical foundations of the Notion-Based Reasoning System (NBRS), a neurosymbolic architecture in which reasoning is performed by traversing a typed directed acyclic graph (DAG) of operations and resolving each node with a small shared neural module. The companion `PROPOSAL.md` document defines a three-track research programme; the present manuscript supplies the formal scaffolding for that programme.

We formalise the central object of study—the *Graph-Structured Recurrent Executor* (GSRE)—as a deterministic state machine over a typed DAG, and prove four foundational properties: (i) the executor preserves an argument-availability invariant; (ii) under a deterministic node processor, the executor is itself deterministic; (iii) the executor terminates in a number of node invocations linear in the size of the graph; (iv) the executor’s output is independent of the topological order in which ready nodes are scheduled. These properties isolate the inductive bias introduced by explicit dependency scheduling from the inductive bias of any particular neural architecture.

We then perform an architectural comparison between GSRE and two natural baselines—a weight-tied graph transformer with causal ancestor masking (B_4 -tied) and a sequence transformer over a serialised DAG (B_5)—and prove an *expressivity containment* result: any function implementable by GSRE is implementable by B_4 -tied, while the converse does not hold. This containment is precisely the source of the inductive-bias differential we aim to measure: GSRE’s smaller hypothesis class encodes the prior that computation must respect dependencies, whereas B_4 -tied must learn this prior from data.

We give an information-theoretic argument that the GSRE’s per-step memory budget is bounded by the maximum in-degree of any primitive, independent of total graph depth, whereas the sequence-transformer baseline’s per-step budget scales with graph size. From this we derive a capacity–depth decoupling result: under a primitive-accurate node processor with per-step error rate ε , the GSRE executes a depth- d graph correctly with probability at least $(1 - \varepsilon)^d$, with no growth in per-step parameter count.

We provide a falsifiable generalisation framework. Under the assumption that the node processor depends only on local node context (parent values, type, and spec), the expected execution accuracy of GSRE on out-of-distribution depths depends only on the marginal distribution of local contexts—never on the depth itself, except through compounded primitive error. We formulate the Track 1 experimental claim as a directly testable hypothesis comparing OOD generalisation slopes.

For Track 2 we present a Markov decision process (MDP) formalisation of autonomous graph construction, characterise the per-step action space, and discuss the implications for sparse-reward reinforcement learning and Monte Carlo tree search. For Track 3 we articulate the formal obstacles to autonomous primitive invention as four distinct open problems: subgraph abstraction, continual learning of the node processor, semantic verification of invented primitives, and the search-vs.-abstraction tradeoff.

We close with a discussion of connections to prior work on compositional generalisation, length generalisation, scratchpad reasoning, neural module networks, program synthesis, and library learning, and with a careful enumeration of the limitations of the present framework.

Contents

1	Introduction	3
1.1	What this document is, and what it is not	3
1.2	The central question	3
1.3	Structure of the document	3
2	Formal Preliminaries	4
2.1	Types and values	4
2.2	Primitive operations	4
2.3	Notions and notion DAGs	5
2.4	Reasoning tasks and oracle execution	5
3	The Graph-Structured Recurrent Executor	6
3.1	Node states	6
3.2	The traversal engine	7
3.3	Core operational theorems	7
3.4	The node processor as a learning target	9
4	Architectural Comparison: B_4-tied and B_5 Baselines	9
4.1	The B_4 -tied baseline	9
4.2	The B_5 baseline	10
4.3	Expressivity containment	10
4.4	The inductive-bias differential	11
5	Information-Theoretic Analysis	11
5.1	Per-step information budget	11
5.2	Capacity–depth decoupling	12
6	Generalisation Theory	12
6.1	Setup: training and test distributions	13
6.2	Local context invariance	13
6.3	The OOD generalisation gap	13
6.4	Failure modes of the baselines	14
6.5	Constants vs. propagation errors	14
7	The Track 1 Experimental Hypothesis	14
7.1	Formal hypothesis	14
7.2	Decision rules	15
7.3	Ablations as causal probes	15
8	Track 2: A Reinforcement-Learning Formalisation	15
8.1	The graph-construction MDP	15
8.2	Action space cardinality	16
8.3	Sample complexity considerations	16
8.4	Risks and partial mitigations	16
9	Track 3: The Formal Obstacles to Primitive Invention	17
9.1	Subgraph abstraction	17
9.2	Continual learning of the node processor	17
9.3	Verification of invented primitives	18
9.4	Search vs. abstraction	18
9.5	Symbolic-neural interface stability	18

10 Connections to Prior Work	18
11 Limitations	19
12 Conclusion	20
A Notation Summary	21
B A Worked GSRE Trace	21

1 Introduction

1.1 What this document is, and what it is not

This manuscript is the theoretical companion to the `PROPOSAL.md` that defines the Notion-Based Reasoning System (NBRS) research roadmap. The proposal sketches a three-track empirical programme; this document supplies the formal definitions, operational semantics, and structural theorems that justify why the Track 1 experiment is worth running, what its possible outcomes mean, and how the architectural choices in Tracks 2 and 3 connect to standing problems in compositional generalisation, neural program synthesis, and library learning.

It is not a paper of empirical claims. Nothing in this document predicts whether NBRS will outperform its baselines on benchmark suites such as SCAN, COGS, or PCFG-SET. The role of this document is to ensure that whatever the empirical result turns out to be, the architecture’s behaviour can be interpreted against a precise statement of what the architecture is supposed to be, what its inductive biases are, and what alternative architectures it should be compared against. In short, we want our empirical results in Track 1 to be interpretable, and interpretability begins with formal definitions.

1.2 The central question

Modern large language models perform impressive reasoning on many tasks, but they do so by simulating multi-step reasoning through autoregressive token prediction. Empirically, this simulation degrades sharply as compositional depth increases: counting [4], nested arithmetic [5], transitive inference, and structured games [6] all exhibit accuracy collapses with depth or length. Several explanations have been proposed—attention dispersion, positional extrapolation failure, lack of state, latent computation budget—and several remedies have been demonstrated to help—chain-of-thought prompting [7], scratchpad fine-tuning [8], recurrent or universal transformers [9], tool use, and direct calls to symbolic engines.

The NBRS project asks a structural question that cuts across these remedies: *if reasoning is fundamentally compositional—a partial-order of operations rather than a sequence of tokens—then what is the right computational substrate?* The hypothesis under investigation is that the right substrate is one in which (i) operations and their argument dependencies are represented explicitly as a typed DAG; (ii) execution respects those dependencies by hard, architectural constraint rather than soft, learned attention; and (iii) the neural component is small, shared, and applied recurrently across nodes, so that compositional depth does not consume neural parameters or context positions.

This hypothesis is intuitively appealing but theoretically thin. The point of this document is to make the hypothesis precise enough that the Track 1 experiment can falsify it.

1.3 Structure of the document

Section 2 introduces formal preliminaries: types, primitive operations, notions, the notion DAG, well-formedness conditions, and compositional depth. Section 3 defines the GSRE as a labelled

state-machine executor and proves its core operational properties. Section 4 formalises the baselines and proves the expressivity-containment result. Section 5 develops the information-theoretic comparison and proves a capacity–depth decoupling theorem. Section 6 formulates compositional out-of-distribution generalisation in our setting and identifies precisely which architectures can be expected to exhibit it under which conditions. Section 7 states the Track 1 empirical hypothesis as a falsifiable predicate over the formalism. Section 8 introduces the MDP formalisation for Track 2 and discusses sample complexity. Section 9 states the open problems of Track 3 in formal language. Section 10 situates the work in the literature, and Section 11 enumerates the limitations of the present framework.

2 Formal Preliminaries

We work in a typed first-order setting. A reasoning task is a function from a typed input to a typed output, and a reasoning trace is a directed acyclic graph of intermediate typed values produced by applying primitive operations.

2.1 Types and values

Definition 2.1 (Type System). Fix a finite, decidable set $\mathcal{T}$ of *types*. For each $\tau \in \mathcal{T}$ we associate a (possibly infinite) *value domain* $\mathcal{V}_\tau$. We write $\mathcal{V} = \bigsqcup_{\tau \in \mathcal{T}} \mathcal{V}_\tau$ for the disjoint union of all value domains, and we say v has type τ (written $v : \tau$) when $v \in \mathcal{V}_\tau$.

Example 2.2 (Track 1 types). For the Track 1 experiments we instantiate $\mathcal{T} = \{\text{Nat}_{101}, \text{Str}_{\leq 5}, \text{Bool}\}$ with

$$\mathcal{V}_{\text{Nat}_{101}} = \{0, 1, \dots, 100\}, \quad \mathcal{V}_{\text{Str}_{\leq 5}} = \bigcup_{k=0}^5 \Sigma^k, \quad \mathcal{V}_{\text{Bool}} = \{\perp, \top\},$$

where Σ is the lowercase English alphabet plus space.

2.2 Primitive operations

Definition 2.3 (Primitive Operation). A *primitive operation* (or *primitive*) is a tuple

$$\tau = (\text{name}_\tau, \text{in}_\tau, \text{out}_\tau, f_\tau),$$

where name_τ is a symbolic identifier, $\text{in}_\tau = (\tau_1, \dots, \tau_{k_\tau}) \in \mathcal{T}^*$ is a (possibly empty) tuple of input types, $\text{out}_\tau \in \mathcal{T}$ is the output type, and $f_\tau : \mathcal{V}_{\tau_1} \times \dots \times \mathcal{V}_{\tau_{k_\tau}} \rightarrow \mathcal{V}_{\text{out}_\tau}$ is the *semantic function* that the primitive denotes.

The semantic function f_τ defines the ground-truth meaning of the primitive. Note that the semantic function is *external* to any neural model: it is the function we want the model to learn to compute.

Definition 2.4 (Primitive Library). A *primitive library* $\mathcal{L} = \{\tau_1, \dots, \tau_K\}$ is a finite, ordered set of primitives. We write $|\mathcal{L}| = K$. A library is *type-closed* if for every $\tau \in \mathcal{L}$ and every $\tau' \in \text{in}_\tau$ there exists some $\tau'' \in \mathcal{L}$ with $\text{out}_{\tau''} = \tau'$ or τ' is an *input type* of the overall task.

Example 2.5 (Track 1 library). For Track 1 we fix $\mathcal{L}_1 = \{\text{ADD_MOD}, \text{SUB_MOD}, \text{MUL_MOD}, \text{SHIFT_CHAR}, \text{SWAP_CASE}, \text{REVERSE}, \text{COMPARE}, \text{SELECT}, \text{CONST}_{\text{Nat}}, \text{CONST}_{\text{Str}}, \dots\}$ with the obvious semantics. Modular arithmetic is taken with respect to the prime modulus $p = 101$ throughout.

2.3 Notions and notion DAGs

We can now define the central data structure.

Definition 2.6 (Notion). A *notion* (or *operation node*) over a library $\mathcal{L}$ is a tuple

$$n = (\text{id}_n, \tau_n, \text{spec}_n, \text{pa}_n),$$

where id_n is a unique identifier, $\tau_n \in \mathcal{L}$ is the primitive type of the notion, spec_n is an auxiliary specification (e.g., a constant value, a shift offset, a comparison direction), and $\text{pa}_n = (p_1, \dots, p_{k_{\tau_n}})$ is a tuple of parent notion identifiers, one per input slot of τ_n . We say notion n is a *leaf* if $\text{pa}_n = ()$, i.e., τ_n has arity zero (a constant or input primitive).

Definition 2.7 (Notion DAG). A *notion DAG* (or *N-DAG*) over $\mathcal{L}$ is a finite collection $G = (V, r)$ where V is a set of notions, all parent identifiers in $\bigcup_{n \in V} \text{pa}_n$ refer to elements of V , the induced parent relation is acyclic, and $r \in V$ is a distinguished *root* (output) notion.

We adopt the standard graph terminology: for $n \in V$, $\text{pa}(n)$ is its set of direct predecessors and $\text{ch}(n)$ is its set of direct successors. We write $\text{ch}(n) = \{n' \in V : n \in \text{pa}(n')\}$.

Definition 2.8 (Well-Formedness). An N-DAG $G = (V, r)$ is *well-formed* if (i) the parent relation is acyclic; (ii) every reference in pa_n is to a notion in V ; (iii) types match along every edge: if $p_i \in \text{pa}_n$ corresponds to the i -th input slot of τ_n with declared type σ_i , then $\text{out}_{\tau_{p_i}} = \sigma_i$; (iv) every non-leaf notion has exactly k_{τ_n} parents, in the order matching the input slots of τ_n .

Throughout the rest of the paper, all N-DAGs are assumed well-formed unless otherwise stated.

Definition 2.9 (Compositional Depth). Let $G = (V, r)$ be a well-formed N-DAG. The *depth* of a notion $n \in V$ is defined recursively by

$$\text{depth}(n) = \begin{cases} 0 & \text{if } \text{pa}_n = (), \\ 1 + \max_{p \in \text{pa}_n} \text{depth}(p) & \text{otherwise.} \end{cases}$$

The *depth of the graph* G is $\text{depth}(G) := \text{depth}(r)$.

Remark 2.10 (Comparing notion DAGs to syntax trees). A notion DAG generalises an abstract syntax tree by permitting value sharing: a single resolved value may be consumed by multiple downstream operations. Track 1 restricts attention to chain-shaped (i.e., tree- and degenerate) N-DAGs to control confounds, but the formal machinery and operational semantics handle the general DAG case from the outset.

Example 2.11 (A small chain). Consider the task “compute $((3 + 5) - 2) \cdot 7 \bmod 101$ ”. The corresponding notion DAG over $\mathcal{L}_1$ contains five notions: $n_1 = (\text{CONST}_{\text{Nat}}, \text{spec}=3)$, $n_2 = (\text{CONST}_{\text{Nat}}, \text{spec}=5)$, $n_3 = (\text{ADD_MOD}, \text{pa}=(n_1, n_2))$, $n_4 = (\text{CONST}_{\text{Nat}}, \text{spec}=2)$, $n_5 = (\text{SUB_MOD}, \text{pa}=(n_3, n_4))$, $n_6 = (\text{CONST}_{\text{Nat}}, \text{spec}=7)$, $n_7 = (\text{MUL_MOD}, \text{pa}=(n_5, n_6))$. Then $r = n_7$, $\text{depth}(G) = 3$, and the semantic value at the root is $((3 + 5) - 2) \cdot 7 = 42 \pmod{101}$.

2.4 Reasoning tasks and oracle execution

Definition 2.12 (Reasoning Task). A *reasoning task* over $\mathcal{L}$ is a pair (G, σ) , where $G = (V, r)$ is a well-formed N-DAG and $\sigma : V_{\text{leaf}} \rightarrow \mathcal{V}$ is a value assignment to the leaves of G respecting type. The *ground-truth answer* of the task is the value at the root produced by recursive substitution of the primitive semantic functions.

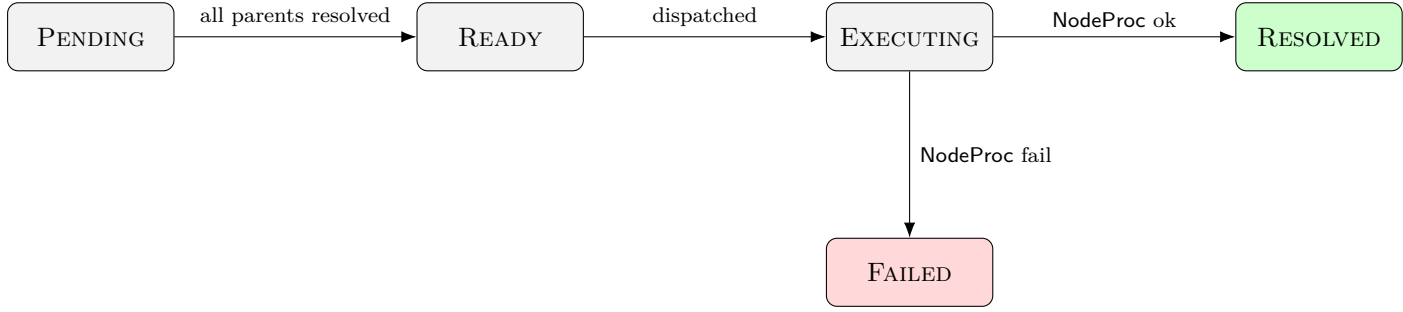


Figure 1: The GSRE node-state machine. A node may transition $\text{PENDING} \rightarrow \text{READY}$ only when all of its parents are in RESOLVED ; $\text{READY} \rightarrow \text{EXECUTING}$ when the scheduler dispatches it; and $\text{EXECUTING} \rightarrow \text{RESOLVED}$ or $\text{EXECUTING} \rightarrow \text{FAILED}$ depending on the outcome of the node-processor call. In the absence of a backtracking policy (Track 1), FAILED is an absorbing terminal state of the graph.

The ground-truth answer can be computed by any topological traversal of G that respects parents-before-children ordering, evaluating each non-leaf notion n as

$$\hat{v}_n = f_{\tau_n}(\hat{v}_{p_1}, \dots, \hat{v}_{p_{k_{\tau_n}}}),$$

where $(p_1, \dots, p_{k_{\tau_n}}) = \text{pa}_n$. The associativity-style argument that makes this well-defined is folklore but worth recording for later reuse:

Lemma 2.13 (Order Independence of Oracle Execution). *Let $G = (V, r)$ be a well-formed N-DAG with leaf assignment σ , and let the primitive semantic functions f_{τ_n} be deterministic. Then for any two topological orderings π_1, π_2 of V , the value $\hat{v}_n$ assigned to each $n \in V$ by oracle execution is the same under π_1 and π_2 .*

Proof. By induction on $\text{depth}(n)$. The base case $\text{depth}(n) = 0$ is immediate because $\hat{v}_n = \sigma(n)$ does not depend on order. For the inductive step, $\hat{v}_n = f_{\tau_n}(\hat{v}_{p_1}, \dots, \hat{v}_{p_{k_{\tau_n}}})$ depends only on the values of its parents, each of which is assigned the same value under π_1 and π_2 by the inductive hypothesis. Determinism of f_{τ_n} then gives equality. $\square$

Lemma 2.13 is what justifies the use of any topological order for ground-truth value generation. It is also the structural reason why GSRE—which is built on the same principle of dependency-respecting execution—is itself order independent (Theorem 3.5).

3 The Graph-Structured Recurrent Executor

We now define the GSRE formally, as a labelled state machine that executes a well-formed N-DAG using a shared neural node processor. The remainder of this section proves the four foundational properties of GSRE promised in the introduction.

3.1 Node states

Definition 3.1 (Node State). At any point during execution, each node $n \in V$ is in one of five states drawn from the set $\mathcal{S} = \{\text{PENDING}, \text{READY}, \text{EXECUTING}, \text{RESOLVED}, \text{FAILED}\}$. We write $\text{state}(n) \in \mathcal{S}$ for the state of n . Once a node enters RESOLVED , its value $v_n \in \mathcal{V}_{\text{out}_{\tau_n}}$ is fixed for the remainder of execution.

Figure 1 depicts the transition relation. The key structural fact is that $\text{PENDING} \rightarrow \text{READY}$ is gated by a hard predicate on parent state, not by a learned signal: the architecture itself enforces the argument-availability invariant.

Algorithm 1 The Graph-Structured Recurrent Executor (GSRE)

Require: Well-formed N-DAG $G = (V, r)$ with leaf assignment σ ; node processor NodeProc_θ .

Ensure: A (possibly partial) value assignment $v : V \rightarrow \mathcal{V}$.

```
1: for each  $n \in V$ :  $\text{state}(n) \leftarrow \text{PENDING}$ 
2: for each leaf  $n \in V$ :  $v_n \leftarrow \sigma(n)$ ;  $\text{state}(n) \leftarrow \text{RESOLVED}$ 
3: loop
4:    $R \leftarrow \{n \in V : \text{state}(n) = \text{PENDING} \text{ and all } p \in \text{pa}_n \text{ have } \text{state}(p) = \text{RESOLVED}\}$ 
5:   for each  $n \in R$ :  $\text{state}(n) \leftarrow \text{READY}$ 
6:   if  $R = \emptyset$  then
7:     if  $\text{state}(r) = \text{RESOLVED}$  then
8:       return  $v$ 
9:     else if  $\text{state}(r) = \text{FAILED}$  or no PENDING node has all-RESOLVED parents then
10:      return  $v$  ▷ stuck
11:     end if
12:   end if
13:    $\mathcal{B} \leftarrow R$  ▷ the GSRE batches all ready nodes for this step
14:   for each  $n \in \mathcal{B}$  (in parallel):  $\text{state}(n) \leftarrow \text{EXECUTING}$ 
15:    $\hat{v}_{\mathcal{B}} \leftarrow \text{NodeProc}_\theta(\{(\tau_n, \text{spec}_n, (v_p)_{p \in \text{pa}_n}) : n \in \mathcal{B}\})$ 
16:   for  $n \in \mathcal{B}$  do
17:     if  $\hat{v}_{\mathcal{B},n} \neq \perp$  then
18:        $v_n \leftarrow \hat{v}_{\mathcal{B},n}$ ;  $\text{state}(n) \leftarrow \text{RESOLVED}$ 
19:     else
20:        $\text{state}(n) \leftarrow \text{FAILED}$ 
21:     end if
22:   end for
23: end loop
```

3.2 The traversal engine

The traversal engine drives the state machine. At each step, it identifies all currently ready nodes, batches them, calls the shared node processor on each, and updates state accordingly. Algorithm 1 gives the formal pseudocode.

Two structural facts are immediate from Algorithm 1.

Lemma 3.2 (Local Input Determinism). *At line 14 of Algorithm 1, the input to NodeProc_θ for any $n \in \mathcal{B}$ consists exclusively of τ_n , spec_n , and the resolved values of pa_n . In particular, this input does not depend on the identity, types, depths, or values of any other node in the graph except via the parent values themselves.*

Proof. By inspection of line 14. □

Lemma 3.3 (Argument-Availability Invariant). *At any point in the execution of Algorithm 1, if n enters state EXECUTING then $\text{state}(p) = \text{RESOLVED}$ for every $p \in \text{pa}_n$.*

Proof. Transition into EXECUTING occurs only at line 13, which is reached only for $n \in \mathcal{B} \subseteq R$, where R was constructed at line 4 to satisfy precisely the predicate that all parents of n are in RESOLVED. Since resolved values are immutable after they are set (line 17), this predicate continues to hold at line 13. □

3.3 Core operational theorems

We now state and prove four foundational properties of GSRE.

Theorem 3.4 (Determinism). *Let NodeProc_θ be a deterministic function of its input set. Then Algorithm 1 is a deterministic algorithm: for any well-formed N -DAG G and any leaf assignment σ , the value assignment v returned by the algorithm is uniquely determined by G , σ , and θ .*

Proof. We prove that the value v_n at every node is uniquely determined. Induction on $\text{depth}(n)$.

Base ($\text{depth}(n) = 0$). Leaves have $v_n = \sigma(n)$ assigned at line 2 of Algorithm 1, which depends only on the input.

Step. Let $\text{depth}(n) > 0$ and suppose every $p \in \text{pa}_n$ has its value uniquely determined. Because G is a DAG and there is no cycle, n eventually appears in the ready set R at some iteration (we will prove termination separately in Theorem 3.6). At the moment it does, line 12 batches n with other ready nodes. Line 14 calls NodeProc_θ with input consisting solely of τ_n , spec_n , and $(v_p)_{p \in \text{pa}_n}$ (Lemma 3.2). These inputs are uniquely determined by G , σ , and θ : τ_n and spec_n from G , and the parent values by the inductive hypothesis. Determinism of NodeProc_θ then fixes v_n uniquely.

The choice of batch composition $\mathcal{B}$ at line 12 does not affect any node's input by Lemma 3.2; the same input is presented regardless of which other ready nodes happen to be batched alongside n . Hence the value assignment is uniquely determined. $\square$

Theorem 3.5 (Order Independence of GSRE). *Under the hypothesis of Theorem 3.4, the value assignment v produced by GSRE is invariant to the order in which ready nodes are dispatched. That is, if line 12 of Algorithm 1 is replaced by an arbitrary nonempty subset $\mathcal{B} \subseteq R$ at each iteration, the eventual value assigned to every node that ever becomes RESOLVED is unchanged.*

Proof. The proof of Theorem 3.4 establishes that the input to NodeProc_θ at a node n is independent of which other nodes are batched alongside it. Hence the output value is the same regardless of batch composition.

Furthermore, every node that can ever become RESOLVED in some schedule does so in every schedule, by the following argument: the set of nodes reachable in RESOLVED depends only on which nodes ever have all parents simultaneously RESOLVED. This is a property of the parent relation, not of the schedule: under a deterministic node processor that never returns $\perp$, every non-leaf node eventually has all parents resolved (induction on depth). When NodeProc_θ may return $\perp$, the set of failed nodes is determined by which nodes' values land on $\perp$, which is itself a function of the (order-independent) inputs. $\square$

Theorem 3.5 is conceptually important: it says that the inductive bias of GSRE is genuinely the one we want—the architecture commits only to dependency-respecting execution, not to any particular traversal order.

Theorem 3.6 (Termination). *Algorithm 1 terminates in at most $|V|$ iterations of the main loop. In particular, the number of NodeProc_θ invocations is at most $|V| - |V_{\text{leaf}}|$.*

Proof. At each iteration, at least one node transitions from PENDING to READY (line 5), provided $R \neq \emptyset$. We argue this is always the case until either r is RESOLVED or every PENDING node has at least one FAILED ancestor.

Consider the iteration when r is not yet resolved and no node has failed. Define the *frontier* $F = \{n \in V : \text{state}(n) = \text{PENDING}\}$. Among F , the nodes whose parents are all in $V \setminus F = \{n : \text{state}(n) = \text{RESOLVED or EXECUTING}\}$ form a nonempty set (this is the standard fact that any nonempty subset of a finite DAG containing at least one descendant-closed element has a minimal element under the induced order, applied to F). These minimal elements are exactly R , which is therefore nonempty.

Thus each iteration either (i) resolves at least one node (when $R \neq \emptyset$), (ii) fails at least one node (which also reduces $|F|$ by removing it from consideration via the stuck check at lines 7–10), or (iii) returns. After at most $|V|$ iterations, every node has been processed.

Per-node, NodeProc_θ is called at most once, so the total number of invocations is at most $|V|$. Leaves are not processed (their values are assigned by line 2), so the bound is in fact at most $|V| - |V_{\text{leaf}}|$. $\square$

Theorem 3.7 (Soundness Against the Oracle). *Let NodeProc_θ be a deterministic node processor that is primitive-correct: that is, for every primitive $\tau \in \mathcal{L}$ and every typed input tuple $(v_1, \dots, v_{k_\tau})$, the output of NodeProc_θ at any node of type τ with parent values $(v_1, \dots, v_{k_\tau})$ equals $f_\tau(v_1, \dots, v_{k_\tau})$. Then for any well-formed N-DAG G with leaf assignment σ , the value v_r returned by GSRE equals the oracle ground-truth answer of Definition 2.12.*

Proof. By induction on $\text{depth}(n)$ together with Lemma 2.13.

Base. For a leaf n , both GSRE and the oracle assign $v_n = \sigma(n)$.

Step. Let $\text{depth}(n) > 0$ and assume that for every $p \in \text{pa}_n$ the GSRE value v_p equals the oracle value $\hat{v}_p$. By primitive correctness of NodeProc_θ , the GSRE output at n is $f_{\tau_n}(v_{p_1}, \dots, v_{p_{k_{\tau_n}}}) = f_{\tau_n}(\hat{v}_{p_1}, \dots, \hat{v}_{p_{k_{\tau_n}}}) = \hat{v}_n$.

Applying the induction to $n = r$ gives $v_r = \hat{v}_r$. $\square$

Theorem 3.7 clarifies the role of the node processor: *the only way GSRE can be wrong is if the node processor is wrong on some primitive in the graph*. This is the source of the capacity–depth decoupling we develop in Section 5.

3.4 The node processor as a learning target

The node processor NodeProc_θ is a small Transformer (in Track 1, two layers, approximately 2M parameters). Its input at a node n consists of a type embedding for τ_n , a spec encoding for spec_n , and the resolved values of the parents pa_n formatted with type-aware delimiters. Its output is a value of type out_{τ_n} , produced through a type-appropriate head (regression for numeric types, classification for finite types, copy for string types).

Crucially:

- The node processor has no access to any global graph embedding.
- It has no access to the index or depth of the node it is processing.
- It has no access to a history vector summarising prior nodes.
- It is invoked once per node per inference, regardless of where that node sits in the graph.

This is what we mean by *local context*: the node processor’s input is a function only of the primitive type, the spec, and the parent values. The implications of this design for generalisation are taken up in Section 6.

4 Architectural Comparison: B_4 -tied and B_5 Baselines

The GSRE alone does not give us a useful claim. We need a baseline whose comparison with GSRE isolates the contribution of explicit dependency scheduling. The Track 1 design specifies two natural choices, and the present section formalises them.

4.1 The B_4 -tied baseline

Definition 4.1 (B_4 -tied: Weight-Tied Graph Transformer with Causal Ancestor Masking). Let $G = (V, r)$ be a well-formed N-DAG with $|V| = N$. Fix a topological ordering $\pi : V \rightarrow \{1, \dots, N\}$, e.g., depth-first preorder. The B_4 -tied baseline computes node values via T stacked

layers, all sharing the same parameters θ :

$$\begin{aligned} h_n^{(0)} &= \text{Embed}(\tau_n, \text{spec}_n) \quad \text{for each } n \in V, \\ h_n^{(t+1)} &= \text{Layer}_\theta(h_n^{(t)}; \{h_{n'}^{(t)} : n' \in \text{Ancestors}_\pi(n)\}), \quad t = 0, \dots, T-1, \\ v_n &= \text{Head}_{\text{out}_{\tau_n}}(h_n^{(T)}), \end{aligned}$$

where $\text{Ancestors}_\pi(n) = \{n' : \pi(n') < \pi(n)\}$, Layer_θ is a self-attention block restricted by an ancestor mask, and Head_σ is a type-specific output head.

The defining structural features of B_4 -tied are: (i) the per-layer parameters θ are shared across t , yielding weight tying analogous to GSRE’s reuse of NodeProc_θ ; (ii) the attention mask in Layer_θ restricts each node to attend only to ancestors in the topological order, preserving the causal structure of the DAG; (iii) all node values are computed in T joint passes rather than node-by-node in dependency order.

The third property is the substantive difference from GSRE. In B_4 -tied, a node’s representation $h_n^{(t)}$ at layer $t > 0$ depends on *layer- t representations of ancestors*, not on the resolved values v_p of parents. There is no architectural enforcement that parent values are computed before child values: the model has T rounds in which to propagate information.

4.2 The B_5 baseline

Definition 4.2 (B_5 : Sequence Transformer Over the Serialised DAG). Let $G = (V, r)$ be a well-formed N-DAG. Let $\text{Serialize}(G)$ be a canonical token sequence representation of G : for each node in topological order, emit a tuple of tokens encoding its identifier, type, spec, and parent identifiers, separated by delimiters. The B_5 baseline computes node values by passing $\text{Serialize}(G)$ through a standard sequence transformer (decoder-only autoregressive, or encoder-decoder) and producing values via type-specific heads at the positions corresponding to each node.

For the present analysis the architectural details of B_5 are immaterial; what matters is that B_5 has the same information available as GSRE and B_4 -tied, but presented as a flat token sequence with no explicit DAG structure.

4.3 Expressivity containment

Proposition 4.3 (Expressivity Containment). *For every choice of GSRE parameters θ_{GSRE} there exists a choice of B_4 -tied parameters θ_{B_4} and depth $T \geq \text{depth}(G)$ such that for every well-formed N-DAG G with depth at most T , the value assignment produced by GSRE with θ_{GSRE} equals the value assignment produced by B_4 -tied with θ_{B_4} .*

Proof sketch. Set $T \geq \text{depth}(G)$. Design the B_4 -tied layer so that, at layer t , a node n with $\text{depth}(n) = t$ first attends to its parents (already resolved at layer t by the inductive construction), copies their values into its working state, and applies $\text{NodeProc}_{\theta_{\text{GSRE}}}$ to compute its own value. At all layers $t' > t$ the node passes its value through unchanged. The ancestor mask permits this attention pattern because parents are ancestors. Crucially, this construction uses the freedom to design layer behaviour as a function of node depth, which can itself be inferred from the masking pattern and position.

Conversely, B_4 -tied has degrees of freedom that GSRE does not: nothing in the architecture prevents a layer from blending in information from non-parent ancestors, nor from updating a node’s representation based on layer- t values of nodes whose true semantic values have not yet been computed (because those nodes’ parents are still “in flight” at layer t). These additional functions are realisable by B_4 -tied but not by GSRE. $\square$

Proposition 4.3 encodes the key intuition: GSRE is a strictly more constrained function class than B_4 -tied. This is a feature, not a bug. The hypothesis under test is whether the strict constraint (dependency-respecting computation) is a sufficiently helpful inductive bias to outweigh the loss in expressivity.

4.4 The inductive-bias differential

We summarise the differences in a single table. Let L denote the maximum length of the value-encoding tokens for any single resolved value, N the number of nodes in the graph, d the depth of the graph, and k the maximum in-degree of any primitive.

Property	Gsre	B_4 -tied	B_5
Per-node input length	$\mathcal{O}(kL)$	$\mathcal{O}(NL)$ via attention	$\mathcal{O}(NL)$ (full seq)
Parameters/depth	shared, fixed	shared, fixed	shared, fixed
Dependency enforcement	hard, gated	soft, learned	soft, learned
Sees node identity	no	yes, via position	yes, via position
Sees graph topology	only via parent values	yes, via mask	yes, via tokens

5 Information-Theoretic Analysis

In this section we make precise two claims about GSRE: (i) its per-step memory budget is bounded by the maximum in-degree of any primitive, independent of the depth of the graph; and (ii) under a primitive-accurate node processor with per-primitive error rate ε , the executor produces the correct root value with probability at least $(1 - \varepsilon)^d$ for a depth- d graph. We refer to these results collectively as *capacity–depth decoupling*.

5.1 Per-step information budget

Let $\text{Enc} : \mathcal{V} \rightarrow \{0, 1\}^*$ be a fixed encoding of values into bit strings, and let $|\text{Enc}(v)|$ denote the length of the encoding of v . Define the *primitive description length* of a node n as

$$\ell(n) = |\text{Enc}(\tau_n)| + |\text{Enc}(\text{spec}_n)| + \sum_{p \in \text{pa}_n} |\text{Enc}(v_p)|.$$

Proposition 5.1 (Per-Step Bounded Input). *The input length of the node processor at any node n is exactly $\ell(n)$, and*

$$\ell(n) \leq |\text{Enc}(\tau_n)| + |\text{Enc}(\text{spec}_n)| + k_{\tau_n} \cdot \max_{v \in \mathcal{V}} |\text{Enc}(v)|.$$

In particular, if $k_{\max} := \max_{\tau \in \mathcal{L}} k_{\tau}$ and $L_{\max} := \max_{v \in \mathcal{V}} |\text{Enc}(v)|$ are both finite, then the per-step input length is bounded by $\mathcal{O}(k_{\max} L_{\max})$, independent of $|V|$ and $\text{depth}(G)$.

Proof. The input enumeration follows from the construction of the node-processor call at line 14 of Algorithm 1; Lemma 3.2 guarantees that no other content appears in the input. The bound follows by replacing each parent value’s encoding by the worst case. $\square$

By contrast, B_5 has per-step input of length $\Theta(NL_{\max})$ (the entire serialised DAG), and B_4 -tied’s per-layer attention is over all ancestors, which in the worst case is $\Omega(d)$ nodes per layer. Per-step computational cost is therefore qualitatively different across the three architectures.

5.2 Capacity–depth decoupling

We now make explicit the consequence of bounded per-step input for depth scaling.

Theorem 5.2 (Capacity–Depth Decoupling under i.i.d. Primitive Error). *Let NodeProc_θ be a stochastic node processor whose error events at distinct primitive invocations are independent. Let ε be a uniform upper bound on the probability that NodeProc_θ returns a value different from the oracle f_{τ_n} at any single node (conditional on receiving the oracle parent values). Then for any well-formed N-DAG $G = (V, r)$ of depth d with no shared subexpressions on the path to r ,*

$$\mathbb{P}[v_r = \hat{v}_r] \geq (1 - \varepsilon)^{|V| - |V_{\text{leaf}}|} \geq (1 - \varepsilon)^d \cdot \delta(G),$$

where $\delta(G)$ accounts for non-path nodes and is $\delta(G) = (1 - \varepsilon)^{|V| - |V_{\text{leaf}}| - d}$. The per-step parameter count of NodeProc_θ remains constant in d .

Proof. The event $v_r = \hat{v}_r$ is implied by the joint event that every internal node n satisfies $v_n = \hat{v}_n$. Conditional on all internal nodes having received the correct parent values (i.e., on every parent being correctly resolved), each internal node’s NodeProc_θ call independently produces the correct value with probability at least $1 - \varepsilon$. Iterating, we obtain

$$\mathbb{P}[v_n = \hat{v}_n \text{ for all internal } n] \geq (1 - \varepsilon)^{|V| - |V_{\text{leaf}}|},$$

which implies the first inequality. The second follows by separating out the at-most- d internal nodes on the path from the lowest leaf to r from the rest.

The claim about per-step parameter count is immediate: NodeProc_θ is the same network at every node, with parameter count independent of $|V|$ and d . $\square$

Theorem 5.2 is a positive result for GSRE: if you can drive the per-primitive error rate ε low enough, depth growth degrades performance gracefully—geometrically, not catastrophically—and at no cost in parameters.

Remark 5.3 (Why the same theorem does not hold for B_5). A sequence transformer trained on length- L_{train} serialised DAGs and tested on length- L_{test} DAGs faces three coupled problems: (i) positional extrapolation, since position embeddings must address indices not seen during training [10]; (ii) attention dispersion, since the per-step input is longer and the attention weights spread [4]; (iii) loss of independence between primitive evaluations, since the model is computing all node values in a coupled latent computation. There is no analogue of Theorem 5.2 for B_5 , and indeed empirically B_5 -like models degrade non-geometrically with length [6].

Remark 5.4 (Why the same theorem holds only conditionally for B_4 -tied). B_4 -tied does have shared parameters across layers, and depth in the architectural sense (number of layers T) is decoupled from depth in the graph sense (longest path d) only if $T \geq d$. If T is fixed at training time, B_4 -tied will fail to produce correct values for graphs of depth $> T$ regardless of training, because the architecture has insufficient layers to propagate information through. Even with sufficient T , B_4 -tied’s per-step computation involves attention over $\Omega(d)$ ancestors per layer, which is qualitatively the same length-extrapolation problem as B_5 faces, albeit mitigated by the ancestor mask.

6 Generalisation Theory

We now turn to the central question: under what conditions can a model trained on shallow graphs be expected to generalise to deeper graphs? The answer for GSRE is clean; the answer for the baselines is conditional on properties that may not hold.

6.1 Setup: training and test distributions

Let $\mathcal{G}_d$ denote the set of well-formed N-DAGs over $\mathcal{L}$ of depth exactly d , and let $\mathcal{D}_d$ be a probability distribution over $\mathcal{G}_d$ together with a leaf-value distribution. The Track 1 design fixes $\mathcal{D}_{\text{train}} = \frac{1}{3} \sum_{d=1}^3 \mathcal{D}_d$ and considers test distributions $\mathcal{D}_d$ for $d \in \{1, 2, 3, 4, 5, 7, 9, 10\}$. Depths 1–3 are in-distribution; depths 4–10 are out-of-distribution along the depth axis.

6.2 Local context invariance

The structural fact about GSRE that yields its generalisation guarantee is invariance to depth at the level of the node processor’s input.

Definition 6.1 (Local Context). The *local context* of a node n in a well-formed N-DAG G with leaf assignment σ is the tuple

$$\text{Ctx}(n) = (\tau_n, \text{spec}_n, (v_p)_{p \in \text{pa}_n}).$$

Lemma 3.2 states that the node-processor input at n equals $\text{Ctx}(n)$. Consequently:

Proposition 6.2 (GSRE Sees Only Local Contexts). *Let G, G' be two well-formed N-DAGs with nodes $n \in G$ and $n' \in G'$ such that $\text{Ctx}(n) = \text{Ctx}(n')$ (equality of types, specs, and parent value tuples). Then the values produced by GSRE at n and n' are identical.*

Proof. Immediate from Lemma 3.2 and the determinism of NodeProc_θ . $\square$

Proposition 6.2 has a powerful consequence for OOD generalisation:

Theorem 6.3 (Local-Context OOD Generalisation). *Let $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}$ be two distributions over well-formed N-DAGs, and let $\mathcal{M}_d$ denote the marginal distribution over local contexts $\text{Ctx}(n)$ induced by drawing a graph from $\mathcal{D}_d$ and then drawing an internal node uniformly. If $\mathcal{M}_{\text{train}} = \mathcal{M}_{\text{test}}$, then the per-node error rate of the GSRE node processor is the same under $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$.*

Proof. The per-node error of the node processor is a function only of $\text{Ctx}(n)$. The expected per-node error under a graph distribution is the integral of this function against $\mathcal{M}_d$. If the marginals agree, the expected errors agree. $\square$

Theorem 6.3 is the formal statement of why GSRE is designed to generalise to deeper graphs: depth shows up in the depth of the DAG and in the number of times the node processor is invoked, but not in the per-invocation marginal distribution of local contexts, so long as the leaf values, types, and specs are drawn from comparable distributions.

6.3 The OOD generalisation gap

Combining Theorem 5.2 and Theorem 6.3 yields a clean expression for the expected exact-match accuracy of GSRE as a function of depth.

Corollary 6.4 (OOD Accuracy under Local-Context Marginal Invariance). *Under the hypotheses of Theorem 5.2 and Theorem 6.3, the expected exact-match accuracy of GSRE on a depth- d test distribution $\mathcal{D}_d$ satisfies*

$$\text{Acc}_{\mathcal{D}_d}(\text{GSRE}) \geq \mathbb{E}_{G \sim \mathcal{D}_d} \left[(1 - \varepsilon)^{|V_G| - |V_{G, \text{leaf}}|} \right].$$

Proof. Theorem 6.3 guarantees that the per-node error rate at any node, including those in test graphs of unfamiliar depth, is bounded above by the training per-node error rate ε . The chain of independent primitive evaluations from Theorem 5.2 then bounds the joint correctness probability. $\square$

For Track 1’s chain-shaped graphs of depth d with no shared subexpressions and no degenerate constant-only sub-paths, $|V_G| - |V_{G, \text{leaf}}| = d$, so the lower bound simplifies to $(1 - \varepsilon)^d$.

6.4 Failure modes of the baselines

Why might B_4 -tied or B_5 fail to exhibit the same depth invariance?

B_4 -tied: ancestor scope grows with depth. While B_4 -tied is constrained to attend only to ancestors, the number of ancestors of a node grows with depth. At a depth- d node in a chain, the ancestor set has size $\Theta(d)$. The attention computation aggregates over this set, and the marginal distribution of *attention input sets* under $\mathcal{D}_d$ shifts with d . Unlike GSRE, B_4 -tied does not have a fixed-size per-step input.

B_5 : full sequence length grows. The serialised DAG length grows with $|V|$, which grows with d . Sequence position embeddings, attention dispersion, and softmax temperature interact with the sequence length in well-documented ways [10, 4].

In both cases the marginal distribution of inputs to the network shifts with depth, so the local-context invariance argument breaks down. Whether the breakdown is empirically significant is exactly what Track 1 measures.

6.5 Constants vs. propagation errors

A technical caveat to Theorem 6.3: the assumption that $\mathcal{M}_{\text{train}} = \mathcal{M}_{\text{test}}$ requires control over the value distributions of resolved parents at depth d . If primitive functions induce non-uniform value distributions on their outputs (e.g., the modular sum of two uniform values is uniform, but the modular product is not), the marginal value distributions at depth- d nodes will differ from those at depth-1 nodes. This is the technical reason the Track 1 design fixes a uniform leaf-value range and uses prime-modulus arithmetic: it preserves the marginal distribution of intermediate values, holding $\mathcal{M}_d$ approximately constant in d .

Remark 6.5 (Why a prime modulus). For modular arithmetic over $\mathbb{Z}/p\mathbb{Z}$ with p prime, $(\mathbb{Z}/p\mathbb{Z})^\times$ is a cyclic group of order $p-1$. The induced distribution on outputs of ADD_MOD, SUB_MOD, and MUL_MOD from independent uniform inputs on $\mathbb{Z}/p\mathbb{Z}$ is itself uniform on $\mathbb{Z}/p\mathbb{Z}$ (or on $(\mathbb{Z}/p\mathbb{Z})^\times$ for multiplicative cases conditioned on nonzero inputs). Hence $\mathcal{M}_d$ is approximately invariant in d .

7 The Track 1 Experimental Hypothesis

We now state the Track 1 hypothesis in formal language, and articulate the conditions under which the experiment falsifies it.

7.1 Formal hypothesis

Hypothesis 7.1 (Track 1 Main). There exists a positive constant $c > 0$ such that, holding training compute, parameter count, and primitive coverage constant, the expected OOD generalisation slope of GSRE across depths $\{5, 7, 9, 10\}$ is at least c greater than that of B_4 -tied across the same depths.

The *OOD generalisation slope* is defined as

$$\text{Slope}(\mathcal{A}) = \frac{\sum_{d \in \{5, 7, 9, 10\}} \text{Acc}_{\mathcal{D}_d}(\mathcal{A})}{4} - \text{Acc}_{\mathcal{D}_3}(\mathcal{A}),$$

where $\mathcal{A}$ is the trained model and $\text{Acc}_{\mathcal{D}_d}(\mathcal{A})$ denotes its exact-match accuracy on the depth- d evaluation set. A more positive slope means more graceful degradation with depth.

7.2 Decision rules

- If $\text{Slope}(\text{Gsre}) - \text{Slope}(B_4\text{-tied}) > c$ with 95% confidence over 5 seeds, Hypothesis 7.1 is **supported**.
- If the difference is statistically indistinguishable from zero, the null hypothesis is **not rejected**: implicit propagation through weight-tied attention masking may be sufficient, and explicit scheduling provides no measurable additional benefit.
- If the difference is negative, Hypothesis 7.1 is **refuted**, and the explicit scheduling architecture imposes a measurable cost relative to its closest implicit counterpart. This outcome would be a meaningful contribution: it would suggest that the benefits of dependency structure are already captured by ancestor masking in a weight-tied transformer.

7.3 Ablations as causal probes

The Track 1 design supplements the primary comparison with three ablations:

- B_4 **unshared** vs. B_4 -**tied** isolates the contribution of weight tying within the implicit-propagation family.
- **Gsre-unshared** vs. **Gsre** isolates the contribution of weight tying within the explicit-scheduling family.
- B_5 vs. B_4 -**tied** isolates the contribution of dependency-aware attention masking.

Together with the primary comparison Gsre vs. B_4 -tied, the four pairwise contrasts span the design space of (dependency scheduling) $\times$ (weight tying) $\times$ (DAG-aware input).

8 Track 2: A Reinforcement-Learning Formalisation

In Track 2 the oracle expansion grammar is removed: the system must construct the reasoning DAG itself from a known primitive library. We formalise this as a Markov decision process and discuss its structural properties.

8.1 The graph-construction MDP

Definition 8.1 (Graph-Construction MDP). The graph-construction MDP for a task specification q over library $\mathcal{L}$ is the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

State. $\mathcal{S}$ is the set of pairs (G, q) where G is a partial well-formed N-DAG over $\mathcal{L}$ together with a partial value assignment v , and q is the task specification. The initial state s_0 is $(\emptyset, q)$.

Action. An action is a triple $a = (n_{\text{select}}, \tau, b)$:

- n_{select} is either a PENDING node with unsatisfied arguments in G (an *expansion target*), or a fresh node identifier (a *new sub-goal*). The first action is necessarily a new sub-goal targeting the task output.
- $\tau \in \mathcal{L}$ is a primitive whose output type matches the demanded type at n_{select} .
- b is a binding of each of τ 's input slots to either an existing resolved value in G 's value pool, or a new sub-goal node of the demanded type.

A special action $a = \text{FAIL}$ terminates the episode.

Transition. $P((G, q), a)$ deterministically produces a new graph G' that incorporates the chosen primitive and bindings, and runs the Gsre traversal engine to resolve any newly-ready nodes. Note that the transition is itself a partial execution of the underlying Gsre.

Reward. $R(s, a, s')$ is 0 unless s' contains a fully-resolved root whose value matches the task's ground-truth answer, in which case $R = +1$. **FAIL** yields $R = 0$.

Discount. $\gamma \in (0, 1)$.

A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ defines a distribution over actions in each state. The *Graph Policy Network* (GPN) parameterises such a policy.

8.2 Action space cardinality

Proposition 8.2 (Per-Step Action Space Size). *At a state $s = (G, q)$ with m unfilled slots across all PENDING nodes and v resolved values in the value pool, the number of legal actions is bounded by*

$$|\mathcal{A}(s)| \leq (m + 1) \cdot |\mathcal{L}| \cdot (v + |\mathcal{L}|)^{k_{\max}} + 1,$$

where the last +1 accounts for *FAIL*.

Proof. Choose an expansion target: $m + 1$ options (existing slot or new root subgoal). Choose a primitive: at most $|\mathcal{L}|$ options (typically far fewer once type matching is enforced). For each of the up to $k_{\max}$ input slots of the chosen primitive, bind to either an existing resolved value (v choices) or a new sub-goal of the required type ($|\mathcal{L}|$ choices, taking primitives as type proxies). Multiplying gives the bound. $\square$

For Track 2’s initial linear-chain setting with $|\mathcal{L}| \approx 15$, $k_{\max} = 2$, $m \approx 1$ (a single open slot at a time), and v growing modestly through the episode, this is a manageable per-step action space of order 10^2 to 10^3 , well within the regime of standard RL algorithms.

8.3 Sample complexity considerations

The sparse reward structure—+1 only on a correctly resolved root—creates the central challenge of Track 2. Two mitigations are anticipated.

Behaviour-cloning bootstrap. For depths 1–3, where Track 1’s oracle expansion grammar produces ground-truth DAGs, the GPN can be pre-trained by supervised learning on the oracle trajectories. The resulting policy serves as the initial distribution for subsequent reinforcement learning. Formally, the bootstrapping loss is

$$\mathcal{L}_{\text{BC}}(\pi_\phi) = -\mathbb{E}_{(s, a^*) \sim \mathcal{D}_{\text{oracle}}} [\log \pi_\phi(a^* | s)],$$

where a^* is the oracle expansion action at s .

MCTS-augmented training. At inference and during data collection, the GPN’s policy and value head can be used as priors for a Monte Carlo tree search over expansion sequences [23]. The MCTS-improved policy targets become the training signal for π_ϕ , in the style of AlphaZero. Because action transitions are deterministic (Definition 8.1), MCTS over the graph-construction MDP is particularly clean: every rollout is a no-stochasticity simulation of the executor, and value estimates depend only on the leaf policy and the (deterministic) executor.

Remark 8.3 (Why deterministic transitions matter). In stochastic-transition MDPs, MCTS rollouts conflate policy improvement with environment variance reduction. In the graph-construction MDP, the only randomness is in the policy itself; once the policy is fixed and the random seed of any stochastic node processor is fixed, the trajectory is deterministic. This makes the MCTS-AlphaZero recipe a particularly natural fit.

8.4 Risks and partial mitigations

The risks already identified in the proposal map onto concrete formal concerns:

- **Sparse reward.** With +1-only reward, the credit-assignment problem is severe even for depth 5. The mitigations enumerated above (BC bootstrap, MCTS) address this in principle. Adding dense intermediate signals (e.g., reward per correctly-resolved intermediate node) is possible but biases the policy toward greedy expansion and is therefore an avenue of last resort.
- **Scalability.** Proposition 8.2 shows the per-step action space is bounded by $(m+1)|\mathcal{L}|(v+|\mathcal{L}|)^{k_{\max}}$. For general DAGs with high in-degree and many open slots, this grows quickly. Track 2 deliberately restricts attention to chain-shaped tasks; general DAGs are deferred.
- **Dependence on NodeProc $_{\theta}$.** The graph-construction MDP’s transition function calls the node processor to resolve newly-ready nodes. A node processor with error rate ε will inject noise into the transition itself, blurring the policy’s view of the value pool. Track 1’s characterisation of ε as a function of training depth informs the Track 2 design.

9 Track 3: The Formal Obstacles to Primitive Invention

Track 3 envisions a system that autonomously extends its own primitive library. This is unsolved territory. The purpose of the present section is not to propose a solution, but to articulate the open problems in language that connects to standing research in program synthesis, library learning, and continual learning.

9.1 Subgraph abstraction

The first formal problem is the identification of reusable subgraphs in successful traces. Let $\mathcal{T} = \{G_1, G_2, \dots\}$ be a collection of successfully-resolved N-DAGs. A *subgraph abstraction* is a typed parameterised graph template T and an embedding mapping $\iota_i : T \hookrightarrow G_i$ for each $G_i \in \mathcal{T}_T \subseteq \mathcal{T}$, such that ι_i preserves the structure of T up to substitution of free type variables and leaves.

Definition 9.1 (Compositional Compression). A subgraph abstraction T *compresses* the trace collection $\mathcal{T}$ by an amount

$$\Delta(T) = \sum_{i: T \hookrightarrow G_i} (|T_{\text{instances in } G_i}| - 1) \cdot |T| - |T|,$$

i.e., the bits saved by replacing T ’s multiple instances with a single abstract reference, minus the cost of storing T itself.

The search problem is to find T with high $\Delta(T)$. This is closely related to graph mining and pattern frequency [26], to program library learning in DreamCoder [18], and to the wake-sleep methodology of inductive program synthesis. The challenge specific to NBRS is that subgraph templates must be parameterised by *types*, not just by constants, and the type system permits open type variables in the input slots of the abstracted primitive.

9.2 Continual learning of the node processor

Adding a new primitive τ_{K+1} to the library requires the node processor NodeProc $_{\theta}$ to learn it. Continual learning of a shared multi-task transformer is a known hard problem.

Conjecture 9.2 (Compositional Continual Learning is Possible). *For a shared node processor NodeProc $_{\theta}$ trained on a sequence of primitives $\tau_1, \tau_2, \dots$ each constructed by abstraction from successful traces involving previously-learned primitives, there exists a training schedule and a parameter-regularisation scheme under which (i) the per-primitive accuracy on each τ_i remains above $1 - \varepsilon$ for some uniform $\varepsilon > 0$; and (ii) the time to reach $1 - \varepsilon$ accuracy on τ_{K+1} is sublinear in K .*

We are unaware of any general technique that achieves both (i) and (ii) for shared transformer architectures. Approaches in the recent literature—LoRA-style adapters [24], elastic-weight consolidation [25], mixture-of-experts—each give up one of these properties for the other. Reconciling them is an open problem and a precondition for Track 3.

9.3 Verification of invented primitives

If a new primitive τ_{K+1} is induced from data, the system must verify that it computes a useful function. Two interpretations of “useful” are possible:

- **Compression-based verification.** The new primitive is useful if including it in the library reduces the description length of held-out problem solutions by a margin exceeding the cost of carrying its parameters.
- **Semantic verification.** The new primitive must compute a well-defined mathematical function $f_{\tau_{K+1}}$. Without an external oracle, this function is implicit in the parameters of NodeProc_θ , and “correctness” is whatever the node processor produces.

Compression-based verification is realisable and is the metric DreamCoder uses. Semantic verification raises a foundational question: *can the system construct test cases for its own invented primitives?* In principle, abstracted primitives come with example traces from the corpus they were mined from, and these examples can serve as a self-generated test set. In practice, the test set defines correctness, and the system has no separate ground truth.

9.4 Search vs. abstraction

When confronted with a novel problem, the system can either (i) attempt to solve it by search over existing primitives, or (ii) attempt to introduce a new primitive. The choice involves meta-reasoning that depends on the empirical search cost of (i) and the compression value of (ii). We have no principled treatment of this tradeoff; it remains the most open of the open problems of Track 3.

9.5 Symbolic-neural interface stability

A final subtle issue: once the system modifies its own operation schemas, the clean separation between symbolic traversal and neural execution—which is the entire architectural premise—becomes recursive. The traversal engine itself must adapt to new node types it has never seen, and the type system itself may need to extend (new types for new primitive outputs). We expect this recursion to be tractable only by maintaining a fixed type system and restricting primitive invention to be type-conservative (no new types are introduced).

10 Connections to Prior Work

The NBRS programme draws on, and connects to, several lines of research.

Length and compositional generalisation. The SCAN benchmark [1] and its descendants [2, 3] established the empirical reality of the compositional-generalisation gap. Subsequent work explored architectural variants that close the gap on specific benchmarks: tree-structured input [11], transformer architectural modifications [12], training-data augmentation [13]. NBRS is in this tradition but addresses compositional generalisation through structural enforcement rather than data augmentation.

Chain-of-thought and scratchpad reasoning. Wei et al. [7] and Nye et al. [8] demonstrated that providing intermediate reasoning steps as targets dramatically improves multi-step task performance. These methods can be viewed as expressing the reasoning DAG implicitly in sequence form. NBRs makes the structure explicit.

Recurrent and universal transformers. Dehghani et al. [9] introduced weight-tied transformers in which the same layer is applied recurrently. GSRE’s B_4 -tied baseline is a graph-structured analogue. The relationship between weight tying and length generalisation has been investigated empirically by [6].

Neural module networks. Andreas et al. [14] proposed networks whose computation graph is determined by a parse of the input question, instantiating modules from a fixed library. NBRs generalises the underlying idea: the computation graph is an explicit, typed DAG, and the modules are unified into a single shared neural module.

Program synthesis and library learning. Track 2 is a neural program synthesis problem with a known DSL; the literature is vast [15, 16, 17]. Track 3 is a library learning problem, most closely related to DreamCoder [18] and earlier work on inductive program synthesis [19]. The novelty of NBRs in this space is the hybrid neural-symbolic substrate: a learned node processor backed by an explicit traversal engine.

Graph neural networks. The B_4 -tied baseline is structurally related to graph neural networks [20, 21], particularly to message-passing variants. The key difference is that GNN message passing operates without dependency-respecting gating: every node updates at every layer in parallel, regardless of whether its input has stabilised.

Causal masking and order constraints. Causal attention masking has been used in graph transformers [22], but typically to encode a fixed topological order rather than to enforce dependency-respecting execution.

11 Limitations

We close with a careful enumeration of the limitations of the theoretical framework developed here.

Independence assumption in Theorem 5.2. The geometric decay $(1 - \varepsilon)^d$ assumed independence of primitive errors across nodes. Empirically, errors at correlated input values (e.g., near-edge cases of modular arithmetic) may be correlated, yielding worse depth scaling. The theorem provides a lower bound under the independence assumption; the actual scaling is an empirical question.

Marginal-invariance assumption in Theorem 6.3. The assumption that the marginal distribution of local contexts is preserved across depths requires careful control over leaf-value distributions and primitive choice. The Track 1 design controls these via uniform sampling and prime-modulus arithmetic; in more complex tasks, marginal invariance may fail.

The node-processor error rate ε may itself grow with depth. Even though the marginal distribution of local contexts is invariant in depth, the actual training data emphasises shallow contexts. Subtle over-reliance on co-occurring shallow features could yield an ε that grows with depth. This is what the experimental measurement of OOD slope checks.

The expressivity-containment result (Proposition 4.3) is constructive but not informative about learning. B_4 -tied can implement GSRE, but whether gradient descent in B_4 -tied parameter space finds the GSRE-implementing setting is a question about training dynamics, not expressivity. Track 1’s empirical setup is precisely a test of this learning question.

The MDP formalisation of Track 2 ignores partial observability. In a realistic deployment, the task specification q may be ambiguous (e.g., natural-language goals). Resolving the ambiguity is itself a perception problem, and the MDP we wrote down assumes q is given in a fully-specified form.

Track 3 is a research agenda, not a proposal. The formal language we provided in Section 9 should not be mistaken for a plan to solve the open problems. Each of the four subsections represents a multi-year research direction in its own right.

No discussion of compute cost. We have not analysed the wall-clock cost of GSRE relative to its baselines. In production deployment, the cost of $|V|$ sequential node-processor invocations may exceed the cost of a single transformer pass on the serialised DAG. Whether the depth-generalisation benefit translates into deployment value depends on the cost ratio and the per-step parallelism achievable on accelerator hardware.

12 Conclusion

The Notion-Based Reasoning System rests on a simple structural hypothesis: that reasoning is fundamentally a partial-order computation, and that representing this partial order explicitly—combined with a small recurrent neural module—unlocks compositional depth generalisation that fully-parallel attention architectures struggle to achieve.

We have formalised the central object of the design (the Graph-Structured Recurrent Executor), proved its core operational properties, articulated its expected information-theoretic advantage over implicit-propagation baselines, and stated the Track 1 empirical hypothesis in falsifiable form. We have also formalised the Track 2 reinforcement-learning problem and articulated the open formal problems of Track 3.

Whether the experiment supports or refutes the hypothesis, the value of the framework lies in turning a vague intuition about “structural reasoning” into a definite, testable, comparable claim. The success of the programme will be measured not by whether GSRE “works”, but by what we learn about the interaction between explicit dependency structure and neural function approximation in compositional reasoning.

What changes for the reader. If Track 1 is supported, we will have evidence that hard architectural enforcement of dependency structure provides measurable benefit over implicit attention-based enforcement. If Track 1 is not rejected, we will have evidence that implicit enforcement is sufficient and the architectural complexity of GSRE is unjustified for this regime. Either outcome is informative; both shape the design choices for Track 2 and the prioritisation among the open problems of Track 3.

Approved for implementation: Track 1.

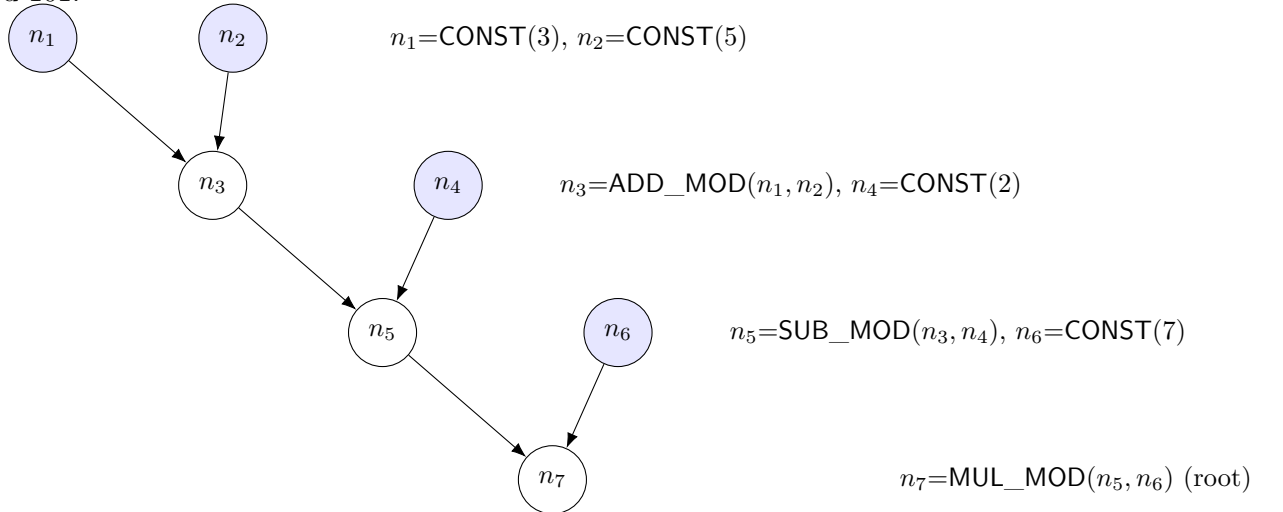
Tracks 2 and 3 remain exploratory and will be refined based on Track 1 outcomes.

A Notation Summary

Symbol	Meaning
$\mathcal{T}$	Set of types in the type system.
$\mathcal{V}_\tau$	Value domain of type τ .
$\mathcal{L}$	Primitive library. $ \mathcal{L} = K$.
τ	A primitive operation; $\text{in}_\tau, \text{out}_\tau, f_\tau$ are its input/output types and semantics.
n	A notion: $(\text{id}_n, \tau_n, \text{spec}_n, \text{pa}_n)$.
$G = (V, r)$	A notion DAG with vertex set V and root r .
$\text{depth}(n)$	Compositional depth of notion n .
$\text{depth}(G)$	Depth of the graph; $= \text{depth}(r)$.
$\text{state}(n)$	Current execution state of $n \in \{\text{PENDING}, \text{READY}, \text{EXECUTING}, \text{RESOLVED}, \text{FAILED}\}$.
v_n	Resolved value at node n .
$\hat{v}_n$	Oracle ground-truth value at node n .
NodeProc_θ	Shared neural node processor with parameters θ .
ε	Per-primitive error rate of NodeProc_θ .
k_τ	Arity of primitive τ . $k_{\max} = \max_\tau k_\tau$.
$\text{Ctx}(n)$	Local context of node n : $(\tau_n, \text{spec}_n, (v_p)_{p \in \text{pa}_n})$.
$\mathcal{D}_d$	Distribution over depth- d N-DAGs.
$\mathcal{M}_d$	Marginal distribution over local contexts under $\mathcal{D}_d$.
GPN_ϕ	Graph Policy Network for Track 2, parameters ϕ .

B A Worked GSRE Trace

For concreteness we trace the execution of GSRE on the DAG of Example 2.11: $((3 + 5) - 2) \cdot 7 \bmod 101$.



The execution trace is:

1. *Initialisation.* All leaves (n_1, n_2, n_4, n_6) are assigned constants and marked RESOLVED with values 3, 5, 2, 7 respectively. All other nodes are PENDING.
2. *Iter 1.* The ready set is $R = \{n_3\}$, since n_3 's parents n_1, n_2 are both RESOLVED. NodeProc_θ is called with input $(\text{ADD_MOD}, \cdot, (3, 5))$. The output is 8. $v_{n_3} = 8$, $\text{state}(n_3) = \text{RESOLVED}$.
3. *Iter 2.* The ready set is $R = \{n_5\}$. NodeProc_θ is called with input $(\text{SUB_MOD}, \cdot, (8, 2))$. The output is 6. $v_{n_5} = 6$, $\text{state}(n_5) = \text{RESOLVED}$.

4. *Iter 3.* The ready set is $R = \{n_7\}$. NodeProc_θ is called with input $(\text{MUL_MOD}, \cdot, (6, 7))$. The output is 42. $v_{n_7} = 42$, $\text{state}(n_7) = \text{RESOLVED}$.
5. *Termination.* The root $r = n_7$ is RESOLVED , so the loop exits and returns v with $v_r = 42$.

In this trace, NodeProc_θ was invoked three times (once per internal node), and at each invocation its input length was bounded by the description length of the primitive plus two parent values—no part of the input depended on the depth of the graph or on the identity of nodes outside the direct parents.

References

- [1] B. M. Lake and M. Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *ICML*, 2018.
- [2] N. Kim and T. Linzen. COGS: A compositional generalization challenge based on semantic interpretation. In *EMNLP*, 2020.
- [3] D. Hupkes, V. Dankers, M. Mul, and E. Bruni. Compositionality decomposed: How do neural networks generalise? *JAIR*, 67, 2020.
- [4] C. Anil et al. Exploring length generalization in large language models. In *NeurIPS*, 2022.
- [5] R. Nogueira, Z. Jiang, and J. Lin. Investigating the limitations of transformers with simple arithmetic tasks. *arXiv:2102.13019*, 2021.
- [6] A. Schwarzschild, E. Borgnia, A. Gupta, F. Huang, U. Vishkin, M. Goldblum, and T. Goldstein. Can you learn an algorithm? Generalizing from easy to hard problems with recurrent networks. In *NeurIPS*, 2021.
- [7] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- [8] M. Nye et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv:2112.00114*, 2021.
- [9] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal transformers. In *ICLR*, 2019.
- [10] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR*, 2022.
- [11] J. Russin, J. Jo, R. C. O’Reilly, and Y. Bengio. Compositional generalization in a deep seq2seq model by separating syntax and semantics. *arXiv:1904.09708*, 2019.
- [12] R. Csordás, K. Irie, and J. Schmidhuber. The devil is in the detail: Simple tricks improve systematic generalization of transformers. In *EMNLP*, 2021.
- [13] J. Andreas. Good-enough compositional data augmentation. In *ACL*, 2020.
- [14] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein. Neural module networks. In *CVPR*, 2016.
- [15] S. Gulwani, O. Polozov, and R. Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2), 2017.
- [16] I. Polosukhin and A. Skidanov. Neural program search: Solving programming tasks from description and examples. *arXiv:1802.04335*, 2018.

- [17] J. Devlin, J. Uesato, S. Bhupatiraju, R. Singh, A.-R. Mohamed, and P. Kohli. RobustFill: Neural program learning under noisy I/O. In *ICML*, 2017.
- [18] K. Ellis et al. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *PLDI*, 2021.
- [19] P. D. Summers. A methodology for LISP program construction from examples. *JACM*, 24(1), 1977.
- [20] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- [21] P. Veličković et al. Graph attention networks. In *ICLR*, 2018.
- [22] V. P. Dwivedi and X. Bresson. A generalization of transformer networks to graphs. *arXiv:2012.09699*, 2020.
- [23] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 2018.
- [24] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [25] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13), 2017.
- [26] T. Washio and H. Motoda. State of the art of graph-based data mining. *ACM SIGKDD Explorations*, 5(1), 2003.